

Rapport de Travaux Pratiques : Sécurisation MQTT via mTLS

Sujet : Mise en place d'une authentification mutuelle TLS (mTLS) avec Mosquitto et OpenSSL

Environnement : WSL

1. Introduction et Objectifs

L'objectif de ce TP est de sécuriser les communications entre un client IoT et un broker MQTT. Contrairement au TLS classique (où seul le serveur est identifié), le **mTLS (mutual TLS)** impose que le client présente également un certificat valide signé par une autorité de confiance (CA).

Objectifs clés :

- Créer une Autorité de Certification (CA) racine.
- Générer et signer des certificats X.509 pour le broker et le client.
- Configurer Mosquitto pour exiger l'authentification par certificat.
- Valider la chaîne de confiance par une publication de message.

2. Architecture de la Chaîne de Confiance

Pour ce TP, nous avons mis en place une hiérarchie de confiance :

1. **CA (ca.crt)** : Le pivot de confiance. Elle sert à signer tous les autres certificats.
2. **Certificat Broker (broker.crt)** : Permet au client de vérifier qu'il parle au bon serveur.
3. **Certificat Client (client.crt)** : Permet au broker de vérifier l'identité du client IoT.

3. Déroulement Technique

Étape A : Génération de l'Autorité de Certification (CA)

Nous avons généré une clé privée RSA de 2048 bits, puis un certificat racine auto-signé valable 10 ans.

```
openssl genrsa -out ca.key 2048
```

```
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt -subj "/CN=MQTT-CA"
```

```
paule@Paule:~/IoT/formation-Jour2$ openssl genrsa -out ca.key 2048
```

```
paule@Paule:~/IoT/formation-Jour2$ openssl req -new -x509 -days 3650 -key ca.key -out ca.crt -subj "/C=FR/ST=IDF/L=Paris/O=IB/CN=MQTT-CA"
```

Étape B : Génération et Signature des certificats

Nous avons créé des demandes de signature (CSR) pour le broker et le client, puis nous les avons signées avec notre propre CA.

- **Broker** : CN=localhost
- **Client** : CN=paule-client

```
paule@Paule:~/IoT/formation-Jour2$ openssl genrsa -out broker.key 2048
paule@Paule:~/IoT/formation-Jour2$ openssl req -new -key broker.key -out broker.csr -subj "/CN=localhost"
paule@Paule:~/IoT/formation-Jour2$ openssl x509 -req -in broker.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out broker.crt -days 365
Certificate request self-signature ok
subject=CN = localhost
```

```
paule@Paule:~/IoT/formation-Jour2$ openssl genrsa -out client.key 2048
paule@Paule:~/IoT/formation-Jour2$ openssl req -new -key client.key -out client.csr -subj "/CN=paule-client"
paule@Paule:~/IoT/formation-Jour2$ openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out client.crt -days 365
Certificate request self-signature ok
subject=CN = paule-client
```

Étape C : Configuration du Broker Mosquitto

Le fichier mosquitto.conf a été configuré pour écouter sur le port standard sécurisé **8883** et exiger un certificat client.

```
paule@Paule:~/IoT/formation-Jour2$ cat mosquitto.conf
listener 8883
cafile ca.crt
certfile broker.crt
keyfile broker.key
require_certificate true
use_identity_as_username false
allow_anonymous true
paule@Paule:~/IoT/formation-Jour2$ |
```

4. Tests et Validation

Le test a été effectué en deux temps :

1. Lancement du broker en mode verbeux : `mosquitto -v -c mosquitto.conf`
2. Publication d'un message depuis un autre terminal avec les certificats.

```

paule@Paule:~/IoT/formation-Jour2$ mosquitto -v -c mosquitto.conf
1774873791: mosquitto version 2.0.11 starting
1774873791: Config loaded from mosquitto.conf.
1774873791: Opening ipv4 listen socket on port 8883.
1774873791: Opening ipv6 listen socket on port 8883.
1774873791: mosquitto version 2.0.11 running
1774873803: New connection from 127.0.0.1:39498 on port 8883.
1774873803: New client connected from 127.0.0.1:39498 as auto-9B2D2C19-6EF7-A92B-A451-12F2AC1FA3FF (p2, c1, k60).
1774873803: No will message specified.
1774873803: Sending CONNACK to auto-9B2D2C19-6EF7-A92B-A451-12F2AC1FA3FF (0, 0)
1774873803: Received PUBLISH from auto-9B2D2C19-6EF7-A92B-A451-12F2AC1FA3FF (d0, q0, r0, m0, 'test', ... (8 bytes))
1774873803: Received DISCONNECT from auto-9B2D2C19-6EF7-A92B-A451-12F2AC1FA3FF
1774873803: Client auto-9B2D2C19-6EF7-A92B-A451-12F2AC1FA3FF disconnected.
^C1774873891: mosquitto version 2.0.11 terminating

```

Commande de test :

mosquitto_pub -h localhost -p 8883 --cafile ca.crt --cert client.crt --key client.key -t "test" -m "Victoire"

```

paule@Paule:~$ cd ~/IoT/formation-Jour2
paule@Paule:~/IoT/formation-Jour2$ mosquitto_pub -h localhost -p 8883
--cafile ca.crt --cert client.crt --key client.key -t "test" -m "Victoire"
paule@Paule:~/IoT/formation-Jour2$

```

5. Résolution des Problèmes (Troubleshooting)

Durant le TP, plusieurs erreurs ont été rencontrées et résolues :

1. **Error: File not found** : Résolue en s'assurant que la commande mosquitto_pub était lancée depuis le dossier contenant les fichiers .crt et .key.

```

paule@Paule:/etc/mosquitto$ mosquitto_pub -h localhost -p 8883 \
> --cafile ca.crt --cert client.crt --key client.key \
> -t "test/secured" -m "mTLS OK"
Error: Problem setting TLS options: File not found.

```

2. **TLS Alert Internal Error** : Cette erreur survient souvent lorsque les certificats ne correspondent pas à la CA actuelle. Une régénération complète de la chaîne a résolu le problème.

```

paule@Paule:~/IoT/formation-Jour2$ mosquitto -v -c mosquitto.conf
1774872615: mosquitto version 2.0.11 starting
1774872615: Config loaded from mosquitto.conf.
1774872615: Opening ipv4 listen socket on port 8883.
1774872615: Opening ipv6 listen socket on port 8883.
1774872615: mosquitto version 2.0.11 running
1774872771: New connection from 127.0.0.1:57524 on port 8883.
1774872771: OpenSSL Error[0]: error:0A000438:SSL routines::tlsv1 alert internal error
1774872771: Client <unknown> disconnected: Protocol error.
^C1774872960: mosquitto version 2.0.11 terminating

```

3. **Connection Refused (Not authorised)** : Résolue en ajustant les paramètres `use_identity_as_username` et `allow_anonymous` dans la configuration.

```

paule@Paule:~/IoT/formation-Jour2$ mosquitto -v -c mosquitto.conf
1774873610: mosquitto version 2.0.11 starting
1774873610: Config loaded from mosquitto.conf.
1774873610: Opening ipv4 listen socket on port 8883.
1774873610: Opening ipv6 listen socket on port 8883.
1774873610: mosquitto version 2.0.11 running
1774873692: New connection from 127.0.0.1:53040 on port 8883.
1774873692: Sending CONNACK to 127.0.0.1 (0, 5)
1774873692: Client <unknown> disconnected, not authorised.
^C1774873779: mosquitto version 2.0.11 terminating

```

6. Déploiement sur GitHub

Le code final a été poussé sur un dépôt GitHub public.

- **Sécurisation** : Utilisation d'un fichier `.gitignore` pour exclure les clés privées (*.key).
- **Automatisation** : Création d'un script `setup.sh` pour automatiser la génération des certificats pour les futurs utilisateurs.
- **Authentification** : Utilisation d'un *Personal Access Token* (PAT) pour le push vers GitHub.

The screenshot shows a GitHub repository named 'IoT' by user 'Paule-2004'. The repository is public and has 1 branch and 0 tags. The file list includes:

File Name	Commit Message	Time
broker.crt	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
broker.csr	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
broker.key	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
ca.crt	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
ca.key	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
client.crt	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
client.csr	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
client.key	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago
mosquitto.conf	IoT exercice_Paule_SAMBA: MQTT mTLS Security Lab	21 minutes ago

There is also a README file. The right sidebar shows repository statistics: 0 stars, 0 watching, 0 forks, and 1 contributor (Paule-2004).